

<StudyMate: 基于大模型的个性化资源生成与学习多智能体系统>

系统部署说明

作者: _____、_____、_____、_____、_____

目 录

- 1 系统概述 1
 - 1.1 项目简介 1
 - 1.2 环境地址 1
 - 1.3 核心功能模块 1
- 2 技术架构 1
 - 2.1 系统架构 1
 - 2.2 技术栈详情 3
- 3 环境要求 3
 - 3.1 硬件配置 3
 - 3.2 软件环境 3
 - 3.3 网络与端口 4
 - 3.4 容量与资源水位 4
- 4 部署流程 4
 - 4.1 部署准备 4
 - 4.2 详细部署步骤 4
 - 4.3 部署完成与访问确认 5
 - 4.4 上线判断与变更边界 5
 - 4.5 配置分级与能力降级 5
- 5 运维与监控 6
 - 5.1 日常维护 6
 - 5.2 数据备份与恢复 6
 - 5.3 故障排查 6
 - 5.4 运维边界与验收留痕 6
 - 5.5 发布后观察与结束处理 6
 - 5.6 比赛展示与故障兜底 7
 - 5.7 最终交付验收清单 7
 - 5.8 部署交接责任与资料去向 7

1 系统概述

1.1 项目简介

StudyMate 是面向高校计算机类课程的个性化资源生成与学习多智能体系统，融合学习画像、混合检索、资源生成、测验评估、语音交互和在线代码执行。

系统覆盖五门计算机课程，七个智能体分别负责检索、讲解、导图、测验、阅读、代码和学习路径，生成内容可保存并追溯来源。

1.2 环境地址

- 开发环境：http://localhost:5173
- 生产环境：https://matropic.cn（已部署，可访问）
- 服务器公网 IP：121.40.64.199；备案号：豫ICP备2026028221号。
- 测试账户：种子库共 19 个账号，包括 1 个管理员、10 个评委和 8 个命名学生。

1.3 核心功能模块

功能模块	核心功能点
用户与权限	邮箱认证：支持注册、登录、退出和会话续期。 角色权限：区分 student、judge、admin。 安全会话：采用 HttpOnly、SameSite 与 Secure Cookie。
学习画像	七维画像：记录基础、目标、薄弱点和学习偏好。 证据更新：结合测验、行为和图片识别。 确认写回：报告经用户确认后更新画像。
课程与 RAG	课程空间：五门课程独立检索与会话。 混合检索：BM25 与 Qwen 向量经 RRF 融合。 引用追溯：展示来源、页码和原文片段。
多智能体生成	Retriever：检索课程证据。 Doc、MindMap、Quiz、Reading、Code：生成学习资源。 Path：规划渐进学习路径并保存工作台。
AI 助教	课程对话：结合上下文和画像多轮问答。 学习模式：支持费曼讲解与苏格拉底追问。 流式与附件：支持 SSE、图片和文档。

功能模块	核心功能点
学习闭环	笔记与错题：支持文件夹、标签和课程归档。 测验报告：记录作答、掌握度和阶段建议。 画像回写：确认后的结果用于后续生成。
可视讲解	主题资源：五门课程共 300 个可视主题。 呈现方式：包含动画、黑板、公式、代码和图表。
语音与拍照	语音能力：支持 ASR 输入和多音色 TTS。 图片能力：支持看图问答、拍照识题和图片转笔记。 附件能力：支持常用文档与代码文件。
拓展与就业	学习资源：接入公开课程和可信阅读详情页。 岗位建议：依据画像、课程和测验生成能力差距建议。
代码与管理	在线编程：运行 Python、C11 和 C++17，并限制资源。 管理功能：支持账号、评委测试、反馈和统计。 终端访问：PC 与手机共用 HTTPS 站点。

2 技术架构

2.1 系统架构

系统采用前后端分离和 Docker Compose 部署：Caddy 提供 HTTPS，Nginx 托管前端并代理 /api，FastAPI 连接 SQLite、课程 RAG、AI 服务和 Piston。

业务数据库、Piston runtime 和 HTTPS 证书分别保存在命名卷中，容器重建与前端更新不会覆盖已有运行数据。

StudyMate 系统架构

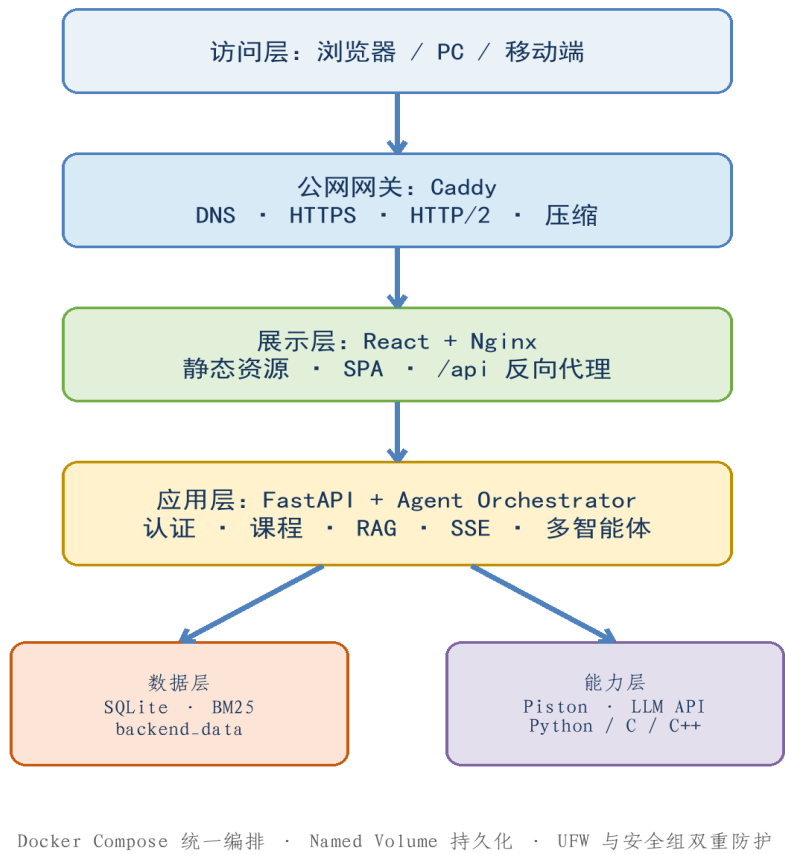


图 2-1 系统架构图

2.1.1 部署形态与适用边界

生产环境采用单机 Docker Compose: Caddy、Frontend、Backend 和 Piston 通过固定容器网络协作，SQLite、runtime 与证书保存在命名卷中。该形态适合竞赛展示和小规模公网访问，不包含多机高可用或自动数据库故障转移。

对象	当前边界
公网入口	仅 Caddy 暴露 80/443，其余服务不直接对公网开放。
数据	线上 backend_data 为权威数据源，镜像种子只初始化空卷。
外部能力	模型、语音、邮件和公开资源异常时按能力降级并明确提示。
扩容	出现持续高并发、SQLite 写锁或单机资源不足后再评估外置服务。

2.2 技术栈详情

分层	技术/组件	核心作用及当前状态
基础设施层	阿里云 ECS / Ubuntu 22.04.5	提供云主机、公网和备案域名。
	Docker 29.6.1 / Compose v5.3.1	编排容器、网络、卷和自动重启。
	Caddy 2	提供 80/443、自动 HTTPS 和压缩。
	Node 20 + Frontend Nginx	完成前端构建、静态托管和 /api 代理。
数据与检索层	SQLite / SQLAlchemy /aiosqlite / Named Volume	保存业务与向量数据，空卷首次播种。
	BM25 + 中英文分词	完成课程关键词召回。
	Qwen Embedding + RRF	融合语义召回与 BM25 排名。
	PostgreSQL / Redis / Chroma	extras 可选预留，主链路未启用。
智能体与模型层	DeepSeek / 星火 / MiMo / Qwen / Qwen-VL	提供生成、助教、视觉和向量能力。
	并发编排 + 7 个资源 Agent	并发生成学习资源和路径。
	讯飞 ASR/TTS + 可选 CosyVoice	提供语音输入与多音色朗读。
应用层（后端）	Python 3.11 + FastAPI 0.115 + Uvicorn 0.32	提供业务、AI、语音和代码 API。
	Pydantic + SSE-Starlette + HTTPX / OpenAI SDK	负责校验、流式事件和外部调用。
	pypdf + multipart + pwdlib /aiosmtplib	支持附件、认证和邮箱验证码。
应用层（前端）	React 19.2.6 + TypeScript 6.0.2 + Vite 8.0.12	构建响应式单页应用。
	Tailwind + Framer Motion + Monaco	实现样式、动画和代码编辑。
	Markdown / KaTeX / Shiki / Markmap / XYFlow / Recharts	展示文档、公式、导图、路径和图表。
代码沙箱层	Piston isolate	在 Docker 内网隔离执行代码。
	Python 3.10 / GCC 10.2 / scikit-learn 1.3.2	运行 Python、C11 和 C++17。
用户层	PC / 手机 Web 浏览器	通过同一 HTTPS 域名访问。

3 环境要求

3.1 硬件配置

配置级别	CPU	内存	SSD	带宽	场景
最低	2 核	4GB	30GB	5Mbps	开发、功能测试和低并发演示
推荐	4 核	8GB	60GB	10Mbps	竞赛评委访问与小规模公网使用
当前	4 vCPU	7.1GiB	59GB	公网带宽	已部署，系统盘约 43GB 可用

当前配置满足竞赛展示与小规模公网访问。

3.2 软件环境

软件/组件	版本/位置	作用
Ubuntu Server	22.04.5 / 宿主机	提供 Docker、SSH 和防火墙环境。
Docker + Compose	29.6.1 / v5.3.1 / 宿主机	管理镜像、容器、网络、卷和重启。
Caddy	2.x / caddy 容器	提供公网 HTTPS、跳转和代理。
Nginx + Node/Vite	alpine、20 / frontend	构建并托管前端，代理 /api/SSE。
Python/FastAPI	3.11 / 0.115 / backend	由 Uvicorn 运行后端 ASGI 服务。
SQLite/SQLAlchemy	2.0.36 / backend_data	持久化业务与向量数据。
Piston	latest / piston-api	隔离运行 Python、C 和 C++。
维护工具	Git/Rsync/Curl/Skopeo/UFW / 宿主机	上传、检查、镜像导入和防火墙维护。
浏览器/DNS/CA	现代浏览器 / 公网	通过正式域名和可信证书访问。

3.3 网络与端口

范围	端口/网络	用途	访问策略
SSH	22/TCP	远程维护	仅可信来源
公网 Web	80/443	ACME、跳转和 HTTPS	公网开放
宿主回环	5173/8000/2000	前端、后端和 Piston 排障	仅 127.0.0.1
容器内网	192.168.242.0/24	核心服务通过名称互访	不经公网
出站	DNS/80/443	镜像、证书、模型和公开资源	按需允许

3.4 容量与资源水位

对象	关注信号	处理原则
CPU/内存	持续高占用、OOM、容器反复重启	区分后端与沙箱负载，保留资源上限后再扩容。
磁盘	镜像、日志、上传、SQLite 和备份持续增长	清理前先列出对象，优先转移备份，禁止自动破坏性 prune。
SQLite	写锁频发、文件快速增长或完整性异常	先优化事务和写入；超过单机边界后评估外置数据库。
外部网络	证书、镜像、模型或语音接口超时	按域名与能力分层定位，不把第三方故障当作数据故障。

4 部署流程

4.1 部署准备

准备 Ubuntu 22.04 服务器和 deploy 用户

确认域名解析及 22/80/443 规则

准备项目、脱敏种子库和服务环境变量

真实密钥仅保存在服务器，文件权限设为 600

Uvicorn、Docker restart/Compose、SQLite 命名卷以及 Caddy + Nginx 分别承担应用运行、进程恢复、数据持久化和公网代理，宿主机无需另装业务运行时。

部署前确认镜像源可用、生产 Profile 已启用，Compose 配置和压缩种子库校验通过。

同时应准备可恢复的数据备份并确认端口边界，避免在缺少回滚点或公网规则不明确时直接上线。

4.2 详细部署步骤

步骤一：以 root/云助手安装 Docker、Compose、Skopeo 和 UFW，完成后重新登录 SSH：

```
DEPLOY_USER=deploy \
bash /home/deploy/studymate-bootstrap.sh
docker --version
docker compose version
```

确认 Docker 为 active、deploy 属于 docker 组；下载失败时按 Docker Hub、GHCR 或依赖源分别排查。

步骤二：生成脱敏 Docker 种子，保留 19 个账号、5 门课程和 1709 个知识块/向量：

```
cd studymate
python3 scripts/build_seed_db.py
gzip -t backend/resources/seed/studymate.db.gz
```

确认数量正确、integrity-check=ok、foreign-key-check=0，且本地运行库未被修改。

步骤三：上传项目，并排除环境变量、本地数据库、依赖、日志和备份：

```
rsync -az --delete --progress \
--exclude '.git/' \
--exclude 'frontend/node_modules/' \
--exclude 'backend/.venv/' \
--exclude 'backend/.env' \
--exclude 'backend/studymate.db' \
--exclude '.deploy.env' \
--exclude 'backups/' --exclude '*.log' \
./studymate/ studymate-server:~/studymate/
```

上传后确认 Compose、Caddyfile、部署脚本和压缩种子存在，服务器环境变量与备份未被覆盖。

步骤四：配置 .deploy.env 与 backend/.env，真实值仅保存在服务器：

```
COMPOSE_PROFILES=public,code-runner
SITE_ADDRESS=matropic.cn
CORS_ORIGINS=https://matropic.cn
SESSION_COOKIE_SECURE=true
chmod 600 backend/.env .deploy.env
```

配置边界：真实模型、语音、邮件和认证密钥不得进入文档、截图、Git 或镜像。

步骤五：GHCR 受限时导入 Piston，并初始化 Python/GCC runtime：

```
cd ~/studymate
bash scripts/import-piston-image.sh
```

Piston 运行时保存在 piston-data，后续更新无需重复下载。

步骤六：一键构建、启动核心服务并初始化运行时：

```
cd ~/studymate
bash scripts/deploy.sh
docker compose --env-file .deploy.env config --quiet
```

步骤七：检查服务、HTTPS、API 与运行时：

```
bash scripts/deploy.sh status
curl -I https://matropic.cn
curl https://matropic.cn/api/ping
curl http://127.0.0.1:2000/api/v2/runtimes
ss -lntp | grep -E ':(80|443|2000|5173|8000)'
```

4.3 部署完成与访问确认

验收对象	检查方法	成功标志	实测
域名与 HTTPS	getent、curl、外网浏览器	解析正确，HTTP 跳转，HTTPS 200	通过
核心容器	compose ps	前后端 healthy，网关/沙箱运行	通过
端口边界	ss、安全组、UFW	仅 80/443 公网	通过
数据完整性	数量、integrity/FK	19 个种子账号、5 门课、1709 块/向量	通过
角色与会话	三类账号登录	权限分离，登录与退出正常	通过
RAG 与 AI	课程问答、SSE	引用正确，流式事件完整	通过
附件/语音/代码	文件、录音、Python/C/C++	解析、ASR/TTS 和运行正常	通过
备案与移动端	PC/手机外网访问	备案可见，页面响应式可用	通过

Caddy 自动申请并续期正式证书；最终应从服务器外部网络完成角色、课程、AI、附件、语音、代码和备案验收。

验收记录应与本次代码版本和数据库备份对应，便于后续复现问题或执行回滚。

4.4 上线判断与变更边界

变更类型	上线前置与必测项	回滚对象
仅前端	生产构建通过，复测首页、登录、/api、移动端和关键交互。	上一版 frontend 镜像。
应用或配置	先备份数据库，校验 Compose，复测健康、权限、SSE 和受影响能力。	后端/网关镜像及上一版配置。
数据或结构	必须有显式迁移、备份副本验证、完整性/外键检查和恢复演练。	迁移与应用版本；必要时独立恢复数据。

上线门禁：缺少可恢复备份、核心容器不健康、数据库异常、权限失效或真实主链路只能返回 mock 时，不得标记为上线完成。

4.5 配置分级与能力降级

配置类别	影响范围	上线要求
入口与认证	域名、HTTPS、CORS、会话和全部登录用户	错误时阻断上线，必须完成登录和 Cookie 验收。
数据与种子	后端启动、线上业务数据和空卷初始化	核对卷、数量、完整性和外键，不允许隐式覆盖。
模型与向量	资源生成、助教、视觉和语义检索	分别验证真实能力与明确降级，mock 不得冒充通过。
语音/邮件/外部资源	单项交互或公开资源推荐	可限制通过，但页面必须清晰提示且文字学习主链路完整。
Piston	在线代码运行	启用时验证 runtime、超时和资源限制；未启用时明确说明。

5 运维与监控

5.1 日常维护

```
cd ~/studymate
bash scripts/deploy.sh status
bash scripts/deploy.sh logs
docker stats --no-stream
docker system df
df -h
sudo ufw status numbered
```

仅更新前端：同步 frontend 后执行 `docker compose --env-file .deploy.env up -d --build frontend`。完整更新：同步代码后执行 `bash scripts/deploy.sh`。

每次更新应记录部署时间和验收结果；即使只修改前端，也要从公网复测首页、登录和 `/api` 代理。

5.2 数据备份与恢复

线上 SQLite 位于 `studymate-backend-data` 卷，更新前使用 backup API 生成一致快照：

```
mkdir -p ~/studymate-backups
docker exec studymate-backend python -c '
import sqlite3
s=sqlite3.connect("/app/data/studymate.db")
d=sqlite3.connect("/app/data/backup.db")
s.backup(d); d.close(); s.close()'
docker cp studymate-backend:/app/data/backup.db \
~/studymate-backups/studymate-$(date +%F_%H%M%S).db
```

恢复前停止 backend 并备份当前库；恢复后检查核心数量、完整性和外键。

数据保护：严禁执行 `docker compose down -v`，该命令会删除数据库、Piston runtime 和 Caddy 证书。

Piston 仅绑定 127.0.0.1，并保留 CPU、内存、PID 和并发限制；日常关注日志、磁盘、容器重启和 SQLite 写锁。

5.3 故障排查

症状	定位方法	处理与禁忌
容器无法启动	config、ps 和容器日志	修复变量或构建，禁止删除卷。
镜像拉取超时	按 Docker Hub、GHCR、依赖源定位	切换对应国内源；Piston 使用导入脚本。
502 Bad Gateway	逐层检查 backend、Nginx、Caddy	修复失败层并重启对应服务。
SQLite 异常	检查数据卷、完整性、外键和写锁	停止写入并在线备份，禁止 <code>down -v</code> 。
AI/SSE/语音异常	检查事件流、日志、出站与配置	恢复网络或变量，不输出真实密钥。
静态或上传异常	检查哈希缓存、代理和大小格式	重建前端或调整文件，不使用 777。
SSL/Piston 异常	检查 DNS、端口、Caddy、runtime API	让 Caddy 重试或重新初始化 runtime。

5.4 运维边界与验收留痕

每次发布应记录代码版本、镜像摘要、配置摘要、数据库备份和最终验收结论。RPO 以上一次已校验且可异机取得的备份为上限；RTO 以实际恢复演练为准，不填写未经验证的固定耗时。

基础设施问题关注服务器、网络、Docker 和磁盘；应用问题关注镜像、配置、接口与日志；数据问题先停止写入并保留现场。

前端更新后复测助教长内容与长截图、四列蛇形路径、雷达标签安全区以及代码标准输入可见性。

模型、语音或公开资源降级必须明确提示；登录、数据完整性、权限和 HTTPS 异常属于上线阻断项。

5.5 发布后观察与结束处理

观察对象	确认内容	异常处理
服务	容器健康、重启次数、错误日志和磁盘余量	保留日志并定位具体服务，不删除命名卷。
业务	登录、课程、RAG、SSE、附件、语音和代码运行	区分核心阻断与可选能力降级，必要时回滚镜像。
数据	用户/课程/知识块数量、完整性、外键和最新备份	停止写入、二次备份，再决定修复或恢复。
安全	证书、公开端口、环境文件权限和测试账号	关闭多余入口，比赛结束后轮换密码与外部凭据。

发布观察完成后，应把最终代码版本、镜像摘要、验收结论和可恢复备份一并归档，作为下一次更新或故障恢复的基线。

5.6 比赛展示与故障兜底

展示环节	正常路径	异常时处理
登录与课程	使用评委账号进入指定课程并确认画像	切换备用测试账号，保留错误信息，不修改线上数据。
AI 与资源生成	展示真实 SSE、引用和工作台资源	明确说明外部模型状态，使用已保存的真实生成记录继续展示。
语音与外部资源	展示已配置能力和可信详情页	单项降级时保留文字学习主链路并给出可解释提示。
在线代码	运行已验证的 Python/C/C++ 示例	Piston 异常时停止重复提交，展示既有结果并在演示后排障。

比赛前应完成一次无调试操作的完整彩排，并准备已验证截图或录屏作为外部服务临时不可用时的辅助材料；兜底材料只能补充说明，不能把 mock 结果描述为真实运行。

5.7 最终交付验收清单

验收项	通过标准
版本	源码、镜像、配置模板和部署文档能够相互对应。
公网	DNS、证书、80/443、备案和外网 PC/手机访问正常。
业务	三类角色、五门课程、RAG、SSE、附件、语音和代码按启用范围通过。
数据	数量符合预期，integrity-check=ok、foreign-key-check=0，备份可取得。
安全	环境文件权限正确，无真实密钥进入源码、文档、截图或交付包。
回滚	保留上一版镜像或代码，明确代码回滚与数据恢复的不同触发条件。
近期界面	长截图、蛇形路径、雷达文字安全区和标准输入显示均完成浏览器回归。

交付前由未参与本次修改的成员按清单独立复核，并把发现的问题、限制条件和最终结论写入验收记录。

5.8 部署交接责任与资料去向

交接对象	资料去向与责任边界	接收方复核
访问与权限	记录云控制台、SSH、域名/DNS 和证书的维护归属，不在文档中保存私钥。	确认能够独立登录，且未共享 root 密码或私钥。
版本与镜像	归档最终 Git 提交、镜像摘要、回滚标签以及本次生成的部署文档。	核对线上容器与交付版本一致。
配置与密钥	仅记录变量名称、服务用途、保管位置和轮换责任人。	确认真实值未进入源码、截图和交付包。
数据与风险	记录 SQLite 实际路径、最近备份、异机副本位置、限制通过项和已知风险。	完成完整性检查，并确认备份可读取和可恢复。

交接完成后，由接收成员独立执行一次容器状态、公网访问、核心业务和数据库检查；复核通过后再把该版本作为下一次更新的维护基线。